

重庆科技大学

《Hadoop 大数据技术实践》 课程设计报告

学 院: 数理科学学院 专业班级: 大数据 23-01

学生姓名: 余东泽 学 号: 2023443661

设计地点 (单位): G406

设计题目: 基于 Hadoop 的淘宝用户行为数据分析

完成日期: 2026 年 1 月 12 日

成绩 (百分制) :

指导教师 (签字): 郭雪荣

课程目标	权重	考核内容	评分标准			
			分值	等级及评分细则	评分区间	实际得分
掌握 Hadoop 技术的基础原理，包括 Hadoop 的基本概念和处理架构等；掌握 Hadoop 存储相关技术的概念与原理，包括分布式文件系统 HDFS、分布式数据库 HBase 的原理及相关编程实践；掌握分布式并行编程模型 MapReduce 的原理及相关编程实践	100%	掌握 Hadoop 生态系统核心组件的基本操作:通过实际操作 HDFS、HBase 和 Hive，深入理解大数据存储、管理和分析的基本原理与方法。培养大数据技术应用能力:通过安装配置、命令操作及应用实例的实践	100	优秀：Hive 环境搭建规范稳定；数据采集完整、预处理逻辑科学；HDFS 导入高效；数据仓库设计合理；Hive SQL 分析深入全面，贴合业务指标；可视化清晰准确；报告完整规范，含关键代码与结果截图，能体现对 Hadoop 技术原理的深刻理解	90-100	
				良好：Hive 环境搭建完成可正常使用；数据采集与预处理基本达标；HDFS 导入顺利；数据仓库设计基本合理；Hive SQL 分析覆盖核心业务指标；可视化基本清晰；报告结构完整，能体现对 Hadoop 技术原理的基本掌握	80-89	
				中等：Hive 环境基本可用，需简单调试；数据采集与预处理完成主要部分；HDFS 导入功能实现；数据仓库设计基本适配需求；Hive SQL 完成基础分析；可视化能呈现核心结果；报告包含核心内容，对 Hadoop 技术原理有初步理解	70-79	
				合格：能搭建基础 Hive 环境；完成数据采集与简单预处理；HDFS 导入基本实现；数据仓库设计简单；Hive SQL 实现简单查询分析；可视化初步完成；报告包含核心模块，了解 Hadoop 基本概念	60-69	
				不合格：无法搭建 Hive 环境；未完成数据采集或预处理；HDFS 导入失败；数据仓库设计缺失；Hive SQL 无法完成有效分析；可视化未实现；报告核心内容缺失，不了解 Hadoop 技术基础原理	0-59	
合计得分						

目录

1. 场景需求分析	4
2. Hive 安装	5
3. 数据源处理与入库	10
4. 数据分析与可视化	13
5. 总结	19

1. 场景需求分析

1.1 业务背景

在数字化电商行业高速发展的背景下，流量获取成本持续攀升，**用户行为分析**已成为电商平台提升运营效率、优化转化路径的核心手段。传统电商运营依赖经验判断，存在“流量转化效率低、用户需求洞察不足、运营策略针对性弱”等痛点：

- 无法量化用户从“浏览商品→加入购物车→完成购买”的全链路转化效率，难以定位转化关键瓶颈；
- 缺乏对用户行为时段规律的分析，无法精准匹配用户活跃时段进行营销推送；
- 海量用户行为日志数据分散存储，传统单机分析工具难以支撑大数据量的高效处理。

本次实验选取某电商平台真实用户行为日志数据作为分析对象，基于 Hadoop 分布式大数据技术栈，构建“数据存储-处理-分析-可视化”的完整链路，旨在通过技术手段解决电商运营的实际业务问题，同时完成分布式系统课程的实践验证。

1.2 分析目标

本次分析分为**业务目标**和**技术目标**两个层面，兼顾电商业务价值与分布式技术实践要求：

1.2.1 业务目标

1. **用户行为分布分析**：统计平台内用户核心行为（浏览 pv、加购 cart、购买 buy）的次数与独立用户数占比，明确平台流量结构，判断核心业务环节的用户参与度；
2. **转化漏斗分析**：计算“浏览→加购”“加购→购买”“浏览→购买”三类核心链路的转化率，定位高转化环节与潜在优化点，为运营策略提供数据支撑；
3. **时段行为趋势分析**：提取用户行为的 24 小时时段特征，识别用户活跃高峰时段，指导平台营销推送、服务器资源调度等业务决策。

1.2.2 技术目标

1. **环境搭建目标**：基于 Docker 容器搭建 Hadoop+Hive 分布式大数据环境，完成 HDFS 分布式存储、Hive 数据仓库、MariaDB 元数据库的配置与验证，掌握分布式环境的部署方法；
2. **数据处理目标**：实现海量用户行为数据的清洗、转换与入库，掌握 Hive SQL 的分布式数据查询与聚合能力，解决单机分析的性能瓶颈；
3. **可视化目标**：通过 Python 可视化工具（Matplotlib）实现分析结果的图表化展示，掌握“大数据分析结果→业务可读图表”的转化方法，提升数据解读效率。

1.3 数据需求

1.3.1 数据源说明

本次分析采用**电商用户行为日志数据**，数据来源于某电商平台的用户操作埋点系统，记录用户在平台内的核心行为轨迹，数据量共计 **971,913 条**，存储格式为文本文件（制表符分隔）。

1.3.2 核心字段需求

数据需包含以下 5 个核心字段，覆盖用户、商品、行为、时间四大维度，字段说明如下表所示：

字段名	字段类型	字段说明	取值示例
user_id	字符串	用户唯一标识，用于区分独立用户	10001082
item_id	字符串	商品唯一标识，关联商品详情信息	28525977507
category_id	字符串	商品分类 ID，用于商品品类分析	4145813
behavior_type	字符串	用户行为类型，限定核心业务行为	pv/cart/buy
behavior_timestamp	长整型	用户行为发生的时间戳（秒级），用于时段分析	1541208000

1.3.3 数据质量要求

为确保分析结果的准确性，数据需满足以下质量标准：

- 1. 完整性：核心字段无缺失值，user_id、behavior_type、behavior_timestamp 不能为空；
- 2. 一致性：behavior_type 仅包含 pv（浏览）、cart（加购）、buy（购买）三类有效值，无异常行为类型；
- 3. 时效性：时间戳覆盖完整的 24 小时时段，确保时段趋势分析的全面性；
- 4. 唯一性：单条日志对应单次用户行为，无重复记录，避免统计结果失真。

1.3.4 数据预处理需求

原始数据需经过以下预处理步骤，方可用于后续分析：

- 1. 格式转换：将 behavior_timestamp 时间戳转换为“年-月-日 时:分:秒”格式，并提取小时字段用于时段分析；
- 2. 异常值清洗：过滤 user_id/item_id 为空、behavior_type 为非有效值的异常记录；
- 3. 数据聚合：按 behavior_type、小时维度进行分组聚合，统计行为次数与独立用户数，降低后续分析的数据量。

2. Hive 安装

使用之前搭建好的 Hadoop 容器，进入 Hadoop 容器

```
yuyuyu@LAPTOP-QSQCCKGKJ: $ docker start 7457fc474fb28da3194237eb3a7d5fa0248a4c5b0a2145efacf6e4686e380252
7457fc474fb28da3194237eb3a7d5fa0248a4c5b0a2145efacf6e4686e380252
yuyuyu@LAPTOP-QSQCCKGKJ: $ docker exec -it 7457fc474fb28da3194237eb3a7d5fa0248a4c5b0a2145efacf6e4686e380252 /bin/bash
* Starting OpenBSD Secure Shell server sshd
root@localhost:/# java -version
openjdk version "1.8.0_462"
OpenJDK Runtime Environment (build 1.8.0_462-8u462-ga~us1-0ubuntu2~22.04.2-b08)
OpenJDK 64-Bit Server VM (build 25.462-b08, mixed mode)
[ OK ]
```

启动 hadoop 和 hbase

```
root@localhost:/# /opt/hadoop/sbin/start-all.sh
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [localhost]
Starting resourcemanager
Starting nodemanagers
root@localhost:/# $HBASE_HOME/bin/start-hbase.sh
localhost: running zookeeper, logging to /opt/hbase/bin/../logs/hbase-root-zookeeper-localhost.out
running master, logging to /opt/hbase/logs/hbase--master-localhost.out
: running regionserver, logging to /opt/hbase/logs/hbase--regionserver-localhost.out
```

查看 hadoop 和 hbase 进程是否正常

```
root@localhost:/# jps
2528 HRegionServer
3200 Jps
468 DataNode
2070 HQuorumPeer
2298 HMaster
715 SecondaryNameNode
1406 NodeManager
270 NameNode
1054 ResourceManager
root@localhost:/# _
```

将 hdfs 命令的可执行文件路径添加到系统的环境变量 PATH 中

```
root@localhost:/# vim /etc/profile
root@localhost:/# source /etc/profile
dirname: missing operand
Try 'dirname --help' for more information.
dirname: missing operand
Try 'dirname --help' for more information.
bash: export: `Hadoop环境变量': not a valid identifier
root@localhost:/# which hdfs
/opt/hadoop/bin/hdfs
root@localhost:/# _
```

测试 HDFS 可写（创建 Hive 所需目录）

```
root@localhost:/# hdfs dfs -mkdir -p /user/hive/warehouse
root@localhost:/# hdfs dfs -mkdir -p /tmp/hive
root@localhost:/# hdfs dfs -chmod 777 /user/hive/warehouse
root@localhost:/# hdfs dfs -chmod 777 /tmp/hive
root@localhost:/# _
```

更新 apt 源（Ubuntu 20.X+ 需更新后才能正常安装包）

```

root@localhost:/# apt update -y
Hit:1 http://archive.ubuntu.com/ubuntu jammy InRelease
Get:2 http://archive.ubuntu.com/ubuntu jammy-updates InRelease [128 kB]
Get:3 http://security.ubuntu.com/ubuntu jammy-security InRelease [129 kB]
Get:4 http://archive.ubuntu.com/ubuntu jammy-backports InRelease [127 kB]
Get:5 http://security.ubuntu.com/ubuntu jammy-security/universe amd64 Packages [1289 kB]
Get:6 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 Packages [3968 kB]
Get:7 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 Packages [1600 kB]
Get:8 http://security.ubuntu.com/ubuntu jammy-security/main amd64 Packages [3637 kB]
Fetched 10.9 MB in 40s (274 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
32 packages can be upgraded. Run 'apt list --upgradable' to see them
root@localhost:/# apt install -y wget vim mariadb-server mariadb-client libmariadb-java

```

安装核心依赖（MariaDB 替代 MySQL，兼容且服务名稳定）

```

root@localhost:/# apt install -y wget vim mariadb-server mariadb-client libmariadb-java
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
vim is already the newest version (2:8.2.3995-1ubuntu2.24).
wget is already the newest version (1.21.2-2ubuntu1.1).
wget set to manually installed.
The following additional packages will be installed:
  galera-4 gawk libfcgi-fast-perl libfcgi-pm-perl libclone-perl libconfig-inifiles-perl libdbd-mysql-perl libdbi-perl
  libencode-locale-perl libfcgi-bin libfcgi-perl libfcgi10db1 libhtml-parser-perl libhtml-tagset-perl
  libhtml-template-perl libhttp-date-perl libhttp-message-perl libio-html-perl liblwp-mediatypes-perl libmariadb3
  libmysqlclient21 libnumal libpopt0 libsigsegv2 libsnappy1v5 libterm-readkey-perl libtimedate-perl liburi-perl
  liburing2 lsof mariadb-client-10.6 mariadb-client-core-10.6 mariadb-common mariadb-server-10.6
  mariadb-server-core-10.6 mysql-common psmisc rsync socat
Suggested packages:
  gawk-doc libmldbm-perl libnet-daemon-perl libsql-statement-perl libdata-dump-perl libipc-sharedcache-perl
  libjna-java libjna-platform-java libbusiness-isbn-perl libwww-perl mailx mariadb-test netcat-openbsd
  python3-braceexpand
The following NEW packages will be installed:
  galera-4 gawk libfcgi-fast-perl libfcgi-pm-perl libclone-perl libconfig-inifiles-perl libdbd-mysql-perl libdbi-perl
  libencode-locale-perl libfcgi-bin libfcgi-perl libfcgi10db1 libhtml-parser-perl libhtml-tagset-perl
  libhtml-template-perl libhttp-date-perl libhttp-message-perl libio-html-perl liblwp-mediatypes-perl libmariadb-java
  libmariadb3 libmysqlclient21 libnumal libpopt0 libsigsegv2 libsnappy1v5 libterm-readkey-perl libtimedate-perl
  liburi-perl liburing2 lsof mariadb-client mariadb-client-core-10.6 mariadb-common mariadb-server
  mariadb-server-10.6 mariadb-server-core-10.6 mysql-common psmisc rsync socat
0 upgraded, 42 newly installed, 0 to remove and 32 not upgraded.
Need to get 21.0 MB of archives.
After this operation, 172 MB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu jammy/main amd64 libpopt0 amd64 1.18-3build1 [28.2 kB]
Get:2 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 rsync amd64 3.2.7-0ubuntu0.22.04.4 [437 kB]
Get:3 http://archive.ubuntu.com/ubuntu jammy/main amd64 libsigsegv2 amd64 2.13-1ubuntu3 [14.6 kB]
Get:4 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 gawk amd64 1:5.1.0-1ubuntu0.1 [447 kB]
Get:5 http://archive.ubuntu.com/ubuntu jammy/main amd64 mysql-common all 5.8+1.0.8 [7212 B]
Get:6 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 mariadb-common all 1:10.6.22-0ubuntu0.22.04.1 [16.5 kB]
Get:7 http://archive.ubuntu.com/ubuntu jammy/universe amd64 galera-4 amd64 26.4.9-1build1 [720 kB]
Get:8 http://archive.ubuntu.com/ubuntu jammy/main amd64 libdbi-perl amd64 1.643-3build3 [741 kB]
Get:9 http://archive.ubuntu.com/ubuntu jammy/main amd64 lsof amd64 4.93.2+dfsg-1.1build2 [253 kB]
Get:10 http://archive.ubuntu.com/ubuntu jammy/main amd64 libconfig-inifiles-perl all 3.0.00003-1 [40.5 kB]
Get:11 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 libmariadb3 amd64 1:10.6.22-0ubuntu0.22.04.1 [188 kB]
Get:12 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 mariadb-client-core-10.6 amd64 1:10.6.22-0ubuntu0.22.04.1 [1002 kB]
Get:13 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 mariadb-client-10.6 amd64 1:10.6.22-0ubuntu0.22.04.1 [1658 kB]
Get:14 http://archive.ubuntu.com/ubuntu jammy/main amd64 libnumal amd64 2.0.14-3ubuntu2 [22.5 kB]
Get:15 http://archive.ubuntu.com/ubuntu jammy/main amd64 libsnappy1v5 amd64 1.1.8-1build3 [17.5 kB]
Get:16 http://archive.ubuntu.com/ubuntu jammy/main amd64 liburing2 amd64 2.1-2build1 [10.3 kB]
Get:17 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 mariadb-server-core-10.6 amd64 1:10.6.22-0ubuntu0.22.04.1 [8070 kB]
Get:18 http://archive.ubuntu.com/ubuntu jammy/main amd64 psmisc amd64 23.4-2build3 [119 kB]
Get:19 http://archive.ubuntu.com/ubuntu jammy/main amd64 socat amd64 1.7.4.1-3ubuntu4 [349 kB]
Get:20 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 mariadb-server-10.6 amd64 1:10.6.22-0ubuntu0.22.04.1 [4128 kB]
Get:21 http://archive.ubuntu.com/ubuntu jammy/main amd64 libhtml-tagset-perl all 3.20-4 [12.5 kB]
Get:22 http://archive.ubuntu.com/ubuntu jammy/main amd64 liburi-perl all 5.10-1 [78.8 kB]
Get:23 http://archive.ubuntu.com/ubuntu jammy/main amd64 libhtml-parser-perl amd64 3.76-1build2 [88.4 kB]
Get:24 http://archive.ubuntu.com/ubuntu jammy/main amd64 libfcgi-pm-perl all 4.54-1 [188 kB]
Get:25 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libfcgi10db1 amd64 2.4.2-2ubuntu0.1 [28.1 kB]
Get:26 http://archive.ubuntu.com/ubuntu jammy/main amd64 libfcgi-perl amd64 0.82+ds-1build1 [22.8 kB]
Get:27 http://archive.ubuntu.com/ubuntu jammy/main amd64 libfcgi-fast-perl all 1:2.15-1 [10.5 kB]
Get:28 http://archive.ubuntu.com/ubuntu jammy/main amd64 libclone-perl amd64 0.45-1build3 [11.0 kB]
Get:29 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libmysqlclient21 amd64 8.0.44-0ubuntu0.22.04.2 [1308 kB]
Get:30 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 libdbd-mysql-perl amd64 4.050-5ubuntu0.22.04.1 [87.6 kB]
Get:31 http://archive.ubuntu.com/ubuntu jammy/main amd64 libencode-locale-perl all 1.05-1.1 [11.8 kB]
Get:32 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libfcgi-bin amd64 2.4.2-2ubuntu0.1 [11.1 kB]
Get:33 http://archive.ubuntu.com/ubuntu jammy/main amd64 libhtml-template-perl all 2.97-1.1 [59.1 kB]

```

创建 Hive 专属元数据库和用户

```

root@localhost:/# mysql -u root -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 44
Server version: 10.6.22-MariaDB-0ubuntu0.22.04.1 Ubuntu 22.04

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> CREATE DATABASE hive_metadata DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci;
USER 'hive'@'localhost' IDENTIFIED BY 'Query OK, 1 row affected (0.001 sec)

MariaDB [(none)]> CREATE USER 'hive'@'localhost' IDENTIFIED BY 'root'; # Hive 连接用户
ALL PRIVILEGES ON hive_metadata.* TO 'hive'@'localhost'; # 授予所有权限
FLUSH PRIVILEGES; # 重载权限
EXIT; #Query OK, 0 rows affected (0.002 sec)

MariaDB [(none)]> GRANT ALL PRIVILEGES ON hive_metadata.* TO 'hive'@'localhost'; # 授予所有权限
Query OK, 0 rows affected (0.002 sec)

MariaDB [(none)]> FLUSH PRIVILEGES; # 重载权限
Query OK, 0 rows affected (0.000 sec)

MariaDB [(none)]> EXIT; # 退出 MariaDB
Bye
root@localhost:/# _

```

下载 hive 并解压

```

root@localhost:/# wget https://archive.apache.org/dist/hive/hive-3.1.3/apache-hive-3.1.3-bin.tar.gz
--2026-01-13 00:43:01-- https://archive.apache.org/dist/hive/hive-3.1.3/apache-hive-3.1.3-bin.tar.gz
Resolving archive.apache.org (archive.apache.org)... 65.108.204.189, 2a01:4f9:1a:a084::2
Connecting to archive.apache.org (archive.apache.org)|65.108.204.189|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 326940667 (312M) [application/x-gzip]
Saving to: 'apache-hive-3.1.3-bin.tar.gz'

apache-hive-3.1.3-bin.tar.gz 100%[=====>] 311.79M 10.7MB/s in 32s

2026-01-13 00:43:37 (9.68 MB/s) - 'apache-hive-3.1.3-bin.tar.gz' saved [326940667/326940667]

```

```

root@localhost:/# tar -zxvf apache-hive-3.1.3-bin.tar.gz -C /opt/
apache-hive-3.1.3-bin/LICENSE
apache-hive-3.1.3-bin/RELEASE_NOTES.txt
apache-hive-3.1.3-bin/NOTICE
apache-hive-3.1.3-bin/binary-package-licenses/com.thoughtworks.paranamer-LICENSE
apache-hive-3.1.3-bin/binary-package-licenses/org.codehaus.janino-LICENSE
apache-hive-3.1.3-bin/binary-package-licenses/org.jamon.jamon-runtime-LICENSE
apache-hive-3.1.3-bin/binary-package-licenses/org.mozilla.rhino-LICENSE
apache-hive-3.1.3-bin/binary-package-licenses/org.jruby-LICENSE
apache-hive-3.1.3-bin/binary-package-licenses/jline-LICENSE
apache-hive-3.1.3-bin/binary-package-licenses/organtlr-LICENSE
apache-hive-3.1.3-bin/binary-package-licenses/org.abego.treelayout.core-LICENSE
apache-hive-3.1.3-bin/binary-package-licenses/sqlite-LICENSE
apache-hive-3.1.3-bin/binary-package-licenses/org.slf4j-LICENSE
apache-hive-3.1.3-bin/binary-package-licenses/org.antlr.antlr4-LICENSE
apache-hive-3.1.3-bin/binary-package-licenses/com.google.protobuf-LICENSE
apache-hive-3.1.3-bin/binary-package-licenses/com.sun.jersey-LICENSE
apache-hive-3.1.3-bin/binary-package-licenses/javolution-LICENSE
apache-hive-3.1.3-bin/binary-package-licenses/NOTICE
apache-hive-3.1.3-bin/binary-package-licenses/asm-LICENSE
apache-hive-3.1.3-bin/binary-package-licenses/com.ibm.icu.icu4j-LICENSE
apache-hive-3.1.3-bin/binary-package-licenses/org.antlr.stringtemplate-LICENSE
apache-hive-3.1.3-bin/examples/files/avro_historical_timestamp_legacy.avro
apache-hive-3.1.3-bin/examples/files/hive_626_bar.txt
apache-hive-3.1.3-bin/examples/files/docurl.txt
apache-hive-3.1.3-bin/examples/files/tjoin2.txt
apache-hive-3.1.3-bin/examples/files/person+age.txt
apache-hive-3.1.3-bin/examples/files/avro_charvarchar.txt
apache-hive-3.1.3-bin/examples/files/parquet_array_null_element.txt
apache-hive-3.1.3-bin/examples/files/smbucket_3.rc
apache-hive-3.1.3-bin/examples/files/tiny_a.txt
apache-hive-3.1.3-bin/examples/files/parquet_type_promotion.txt
apache-hive-3.1.3-bin/examples/files/over10k.gz
apache-hive-3.1.3-bin/examples/files/map_table.txt
apache-hive-3.1.3-bin/examples/files/table1.avsc
apache-hive-3.1.3-bin/examples/files/in9.txt
apache-hive-3.1.3-bin/examples/files/tjoin1.txt
apache-hive-3.1.3-bin/examples/files/sortdp.txt
apache-hive-3.1.3-bin/examples/files/5col_data.txt
apache-hive-3.1.3-bin/examples/files/tsformat.json
apache-hive-3.1.3-bin/examples/files/sample-queryplan-in-history.txt
apache-hive-3.1.3-bin/examples/files/tiny_b.txt
apache-hive-3.1.3-bin/examples/files/non_ascii.tbl.txt

```

验证 hive 是否成功下载并查看版本

```

root@localhost:/# hive --version
Hive 3.1.3
Git git://MacBook-Pro.fios-router.home/Users/ngangam/commit/hive -r 4df4d75bf1e16fe0af75aad0b4179c34c07fc975
Compiled by ngangam on Sun Apr 3 16:58:16 EDT 2022
From source with checksum 5da234766db5dfbe3e92926c9bbab2af
root@localhost:/# _

```

配置 hive-env.sh

```
# export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64
# export HADOOP_HOME=/opt/hadoop
# export HIVE_HOME=/opt/hive
# export HIVE_CONF_DIR=$HIVE_HOME/conf
```

配置 hive-site.xml

```

<!-- 4. *E* 容 Hadoop 3.x *E*M置 *H* *A* *E*M* *P* *J* *J* *T*Y* *I* -->
<property>
  <name>hive.stats.jdbc.timeout</name>
  <value>30000</value>
</property>
<property>
  <name>hive.stats.retries.max</name>
  <value>10</value>
</property>

<!-- Compatibility configuration for MySQL/MariaDB metastore storage -->
<property>
  <name>datanucleus.connection.driverClass</name>
  <value>org.mariadb.jdbc.Driver</value>
</property>
<property>
  <name>datanucleus.schema.autoCreateAll</name>
  <value>>false</value>
  <description>Disable automatic table creation (schema initialization via schematool)</description>
</property>
<property>
  <name>hive.metastore.schema.verification</name>
  <value>>false</value>
  <description>Disable metastore schema version verification (avoids version compatibility issues)</description>
</property>
</configuration>

```

配置日志目录

```
property.hive.log.level = INFO
property.hive.root.logger = DRFA
property.hive.log.dir = property.hive.log.dir = ${HIVE_HOME}/logs_
property.hive.log.file = hive.log
property.hive.perflogger.log.level = INFO
```

初始化 Hive 元数据库

```
root@localhost:~# schematool -dbtype mysql -initSchema -verbose
Metastore connection URL: jdbc:mariadb://localhost:3306/hive_metastata?createDatabaseIfNotExist=true&useSSL=false&
serverTimezone=UTC&allowPublicKeyRetrieval=true
Metastore Connection Driver : org.mariadb.jdbc.Driver
Metastore connection User: hive
org.apache.hadoop.hive.metastore.HiveMetaException: Failed to get schema version.
Underlying cause: java.sql.SQLException: Could not connect to address=(host=localhost)(port=330
6)(type=master) : (conn=33) Access denied for user 'hive'@'localhost' (using password: YES)
Current charset is UTF-8. If password has been set using other charset, consider using option 'passwordCharacterEncoding
SQL Error code: 1045
org.apache.hadoop.hive.metastore.HiveMetaException: Failed to get schema version.
    at org.apache.hadoop.hive.metastore.tools.HiveSchemaHelper.getConnectionToMetastore(HiveSchemaHelper.java:94)
    at org.apache.hive.beeline.HiveSchemaTool.getConnectionToMetastore(HiveSchemaTool.java:169)
    at org.apache.hive.beeline.HiveSchemaTool.testConnectionToMetastore(HiveSchemaTool.java:475)
```

验证 Hive 安装 (核心步骤)

```
hive> create database test_hive;
OK
Time taken: 0.48 seconds
hive> use test_hive;
OK
Time taken: 0.018 seconds
hive> create table student (id int, name string, age int)
> row format delimited fields terminated by '\t'
> stored as textfile;
OK
Time taken: 0.295 seconds
```

查询数据

```
hive> select * from student;
1      zhangsan      20
2      lisi          22
Time taken: 0.081 seconds, Fetched: 2 row(s)
hive>
```

3. 数据源处理与入库

数据文件如下：

名称	修改日期	类型	大小
user_behavior_100w.csv	2026/1/13 9:49	XLS 工作表	37,158 KB
user_behavior_clean_100w.csv	2026/1/13 9:54	XLS 工作表	55,075 KB
UserBehavior.csv	2026/1/13 9:37	XLS 工作表	3,586,277 KB

抽取 100w 条数据代码：

```
import pandas as pd
import os

# 原始数据源路径（处理重复目录名问题）
raw_data_path = r"D:\UserBehavior.csv\UserBehavior.csv"

# 输出路径（100 万条抽样数据）
output_path = r"D:\UserBehavior.csv\user_behavior_100w.csv"

# 读取原始数据（指定编码，避免中文乱码）
df = pd.read_csv(
    raw_data_path,
    encoding="utf-8",
    header=0,
    names=["user_id", "item_id", "category_id", "behavior_type", "timestamp"]
)

# 抽取前 100 万条数据（确保数据量精准）
df_100w = df.head(1000000).copy()

# 保存抽样数据（使用制表符分隔，适配 Hive 导入格式）
df_100w.to_csv(
    output_path,
    sep="\t",
```

```

        index=False,
        encoding="utf-8"
    )
    print(f"数据抽取完成！")
    print(f"原始数据总行数: {len(df)}")
    print(f"抽样数据总行数: {len(df_100w)}")
    print(f"输出路径: {output_path}")

```

```

PS C:\Users\叮> & C:/Users/叮/AppData/Local/Programs/Python/Python39/python.exe d:/qq文件/机器学习/分布式期末hive数据代码.py
数据抽取完成！
原始数据总行数: 100150806
抽样数据总行数: 1000000
输出路径: D:\UserBehavior.csv\user_behavior_100w.csv

```

3.1 清洗数据:

```

import pandas as pd
import datetime
# 读取抽样数据
df = pd.read_csv(
    r"D:\UserBehavior.csv\user_behavior_100w.csv",
    sep="\t",
    encoding="utf-8"
)
print("清洗前数据概况:")
print(f"总行数: {len(df)}")
print(f"缺失值统计: \n{df.isnull().sum()}")
print(f"重复值数量: {df.duplicated().sum()}")
print(f"行为类型分布: \n{df['behavior_type'].value_counts()}")

# 1. 删除缺失值 (关键字段不能为空)
df_clean = df.dropna(subset=["user_id", "item_id", "behavior_type", "timestamp"])
# 2. 删除重复值
df_clean = df_clean.drop_duplicates(subset=["user_id", "item_id", "behavior_type", "timestamp"])
# 3. 时间戳转换为标准时间格式 (新增 behavior_time 字段)
df_clean["behavior_time"] = df_clean["timestamp"].apply(
    lambda x: datetime.datetime.fromtimestamp(x).strftime("%Y-%m-%d %H:%M:%S")
)
# 4. 过滤无效行为类型 (仅保留 pv、cart、collect、buy)
valid_behavior = ["pv", "cart", "collect", "buy"]
df_clean = df_clean[df_clean["behavior_type"].isin(valid_behavior)]
# 保存清洗后的数据
df_clean.to_csv(
    r"D:\UserBehavior.csv\user_behavior_clean_100w.csv",
    sep="\t",
    index=False,
    encoding="utf-8"
)

print("\n清洗后数据概况:")
print(f"总行数: {len(df_clean)}")
print(f"缺失值统计: \n{df_clean.isnull().sum()}")
print(f"重复值数量: {df_clean.duplicated().sum()}")
print(f"行为类型分布: \n{df_clean['behavior_type'].value_counts()}")
print(f"清洗后数据输出路径: D:\\UserBehavior.csv\\user_behavior_clean_100w.csv")

```

```

清洗前数据概况:
总行数: 1000000
缺失值统计:
user_id      0
item_id      0
category_id   0
behavior_type 0
timestamp    0
dtype: int64
重复值数量: 0
行为类型分布:
behavior_type
pv      896106
cart    55447
fav     28088
buy     20359
Name: count, dtype: int64

清洗后数据概况:
总行数: 971912
缺失值统计:
user_id      0
item_id      0
category_id   0
behavior_type 0
timestamp    0
behavior_time 0
dtype: int64
重复值数量: 0
行为类型分布:
behavior_type
pv      896106
cart    55447
buy     20359
Name: count, dtype: int64
清洗后数据输出路径: D:\UserBehavior.csv\user_behavior_clean_100w.csv
PS C:\Users\呀>

```

3.2 建库建表、数据入库:

将数据上传到容器

```

C:\Windows\System32>docker cp D:\UserBehavior.csv\user_behavior_clean_100w.csv 7457fc474fb28da3194237eb3a7d5fa0248a4c5b0a2145efacf6e4686e380252:/root/
Successfully copied 56.4MB to 7457fc474fb28da3194237eb3a7d5fa0248a4c5b0a2145efacf6e4686e380252:/root/
C:\Windows\System32>_

```

将数据文件从容器内上传到 hdfs

```

root@localhost:/# ls -l /root/user_behavior_clean_100w.csv
-rwxr-xr-x 1 root root 56395905 Jan 13 01:54 /root/user_behavior_clean_100w.csv
root@localhost:/# hdfs dfs -put -f /root/user_behavior_clean_100w.csv /user/hive/warehouse/ecommerce.db/user_behavior/
root@localhost:/# hdfs dfs -ls /user/hive/warehouse/ecommerce.db/user_behavior/
Found 1 items
-rw-r--r-- 1 root supergroup 56395905 2026-01-13 02:55 /user/hive/warehouse/ecommerce.db/user_behavior/user_behavior_clean_100w.csv
root@localhost:/# _

```

加载数据到 Hive 表 (Hive 交互模式)

```

hive> USE ecommerce;
OK
Time taken: 0.015 seconds
hive> -- 直接读取容器内 /root 目录的文件, Hive 自动上传到 HDFS
hive> LOAD DATA LOCAL INPATH '/root/user_behavior_clean_100w.csv' OVERWRITE INTO TABLE user_behavior;
Loading data to table ecommerce.user_behavior
OK
Time taken: 0.448 seconds
hive> SELECT COUNT(*) FROM user_behavior;
Automatically selecting local only mode for query
Query ID = root_20260113030407_54b5c5b6-8e2f-4b15-aed3-6d840dd62c02
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks determined at compile time: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Job running in-process (local Hadoop)
2026-01-13 03:04:09,835 Stage-1 map = 100%, reduce = 100%
Ended Job = job_local1078898190_0001
MapReduce Jobs Launched:
Stage-Stage-1:  HDFS Read: 112791826 HDFS Write: 112791916 SUCCESS
Total MapReduce CPU Time Spent: 0 msec
OK
971913
Time taken: 2.527 seconds, Fetched: 1 row(s)
hive>

```

4. 数据分析与可视化

基础行为分布 + 转化漏斗+完成时段行为分析

基础行为分布聚焦用户行为类型的数量与占比，明确浏览、加购、收藏、购买四类行为的结构特征，可量化平台流量基数与转化现状，判断行为健康度 —— 如高浏览低购买则指向转化短板，为后续分析提供基准。转化漏斗则按“浏览→加购→购买”的用户行为链路，通过去重关联计算各环节转化率，精准定位流失瓶颈：若浏览转加购率低，需优化商品吸引力；若加购转购买率低，则要改善下单支付体验。

分步创建行为分布表

第一步：先统计各行为的基础数据（数量、独立用户数）

```

hive> USE ecommerce;
OK
Time taken: 0.014 seconds
hive> set hive.exec.mode.local.auto=false;
hive> CREATE TABLE IF NOT EXISTS behavior_base_stats AS
> SELECT
>   behavior_type,
>   CASE behavior_type
>     WHEN 'pv' THEN '浏览'
>     WHEN 'cart' THEN '加购'
>     WHEN 'collect' THEN '收藏'
>     WHEN 'buy' THEN '购买'
>   END AS behavior_name,
>   COUNT(*) AS behavior_count,
>   COUNT(DISTINCT user_id) AS user_count
> FROM user_behavior
> GROUP BY behavior_type;
Query ID = root_20260113031602_7df51f12-1e3c-4d4f-af9a-60fa8d41f8b2
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks not specified. Estimated from input data size: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Starting Job = job_1768264552658_0001, Tracking URL = http://localhost:8088/proxy/application_1768264552658_0001
Kill Command = /opt/hadoop/bin/mapred job -kill job_1768264552658_0001
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 1
2026-01-13 03:16:10.088 Stage-1 map = 0%, reduce = 0%
2026-01-13 03:16:14.166 Stage-1 map = 100%, reduce = 0%, Cumulative CPU 4.83 sec
2026-01-13 03:16:19.234 Stage-1 map = 100%, reduce = 100%, Cumulative CPU 6.94 sec
MapReduce Total cumulative CPU time: 6 seconds 940 msec
Ended Job = job_1768264552658_0001
Moving data to directory hdfs://localhost:9000/user/hive/warehouse/ecommerce.db/behavior_base_stats
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1 Reduce: 1 Cumulative CPU: 6.94 sec HDFS Read: 56406689 HDFS Write: 173 SUCCESS
Total MapReduce CPU Time Spent: 6 seconds 940 msec
OK
Time taken: 18.34 seconds
hive>

```

第二步：计算总行为数（用于占比计算）

```
hive> CREATE TABLE IF NOT EXISTS total_behavior AS

    > SELECT SUM(behavior_count) AS total_count FROM behavior_base_stats;
Query ID = root_20260113031729_00e03e54-63c7-47e5-98fb-98abef30881d
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks determined at compile time: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Starting Job = job_1768264552658_0002, Tracking URL = http://localhost:8088/proxy/application_1768264552658_0002/
Kill Command = /opt/hadoop/bin/mapred job -kill job_1768264552658_0002
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 1
2026-01-13 03:17:36,580 Stage-1 map = 0%, reduce = 0%
2026-01-13 03:17:41,808 Stage-1 map = 100%, reduce = 0%, Cumulative CPU 3.46 sec
2026-01-13 03:17:45,864 Stage-1 map = 100%, reduce = 100%, Cumulative CPU 4.77 sec
MapReduce Total cumulative CPU time: 4 seconds 770 msec
Ended Job = job_1768264552658_0002
Moving data to directory hdfs://localhost:9000/user/hive/warehouse/ecommerce.db/total_behavior
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1 Reduce: 1 Cumulative CPU: 4.77 sec HDFS Read: 12958 HDFS Write: 87 SUCCESS
Total MapReduce CPU Time Spent: 4 seconds 770 msec
OK
Time taken: 17.385 seconds
hive>
```

第三步：关联计算占比，生成最终行为分布表

```
hive> CREATE TABLE IF NOT EXISTS behavior_distribution AS
    > SELECT
    >     b.behavior_type,
    >     b.behavior_name,
    >     b.behavior_count,
    >     b.user_count,
    >     CONCAT(ROUND(b.behavior_count / t.total_count * 100, 2), '%') AS behavior_ratio
    > FROM behavior_base_stats b
    > CROSS JOIN total_behavior t
    > ORDER BY b.behavior_count DESC;
Warning: Map Join MAPJOIN[13][bigTable=?] in task 'Stage-2:MAPRED' is a cross product
Query ID = root_20260113032523_7f8c4d50-f37c-47f5-8575-27c73c348f01
Total jobs = 1
2026-01-13 03:25:28 Dump the side-table for tag: 1 with group count: 1 into file: file:/tmp/root/723c51e3-14db-4c82-a195-a282feddb13c/hive_2026-01-13_03-25-23_430_4664025019762083356-1/-local-10005/HashTable-Stage-2/MapJoin-mapfile01--.hashtable
2026-01-13 03:25:28 Uploaded 1 file to: file:/tmp/root/723c51e3-14db-4c82-a195-a282feddb13c/hive_2026-01-13_03-25-23_430_4664025019762083356-1/-local-10005/HashTable-Stage-2/MapJoin-mapfile01--.hashtable (281 bytes)
Execution completed successfully
MapredLocal task succeeded
Launching Job 1 out of 1
Number of reduce tasks determined at compile time: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Starting Job = job_1768264552658_0003, Tracking URL = http://localhost:8088/proxy/application_1768264552658_0003/
Kill Command = /opt/hadoop/bin/mapred job -kill job_1768264552658_0003
Hadoop job information for Stage-2: number of mappers: 1; number of reducers: 1
2026-01-13 03:25:36,223 Stage-2 map = 0%, reduce = 0%
2026-01-13 03:25:40,313 Stage-2 map = 100%, reduce = 0%, Cumulative CPU 1.23 sec
2026-01-13 03:25:45,764 Stage-2 map = 100%, reduce = 100%, Cumulative CPU 2.42 sec
MapReduce Total cumulative CPU time: 2 seconds 420 msec
Ended Job = job_1768264552658_0003
Moving data to directory hdfs://localhost:9000/user/hive/warehouse/ecommerce.db/behavior_distribution
MapReduce Jobs Launched:
Stage-Stage-2: Map: 1 Reduce: 1 Cumulative CPU: 2.42 sec HDFS Read: 20788 HDFS Write: 198 SUCCESS
Total MapReduce CPU Time Spent: 2 seconds 420 msec
OK
Time taken: 23.464 seconds
hive>
```

验证分析结果

```
hive> SELECT * FROM behavior_distribution;
OK
pv      浏览      896106      9706      92.2%
cart    加购      55447      7323      5.7%
buy     购买      20359      6689      2.09%
behavior_type NULL      1      1      0.0%
Time taken: 0.078 seconds, Fetched: 4 row(s)
```

导出结果到本地（供可视化使用）

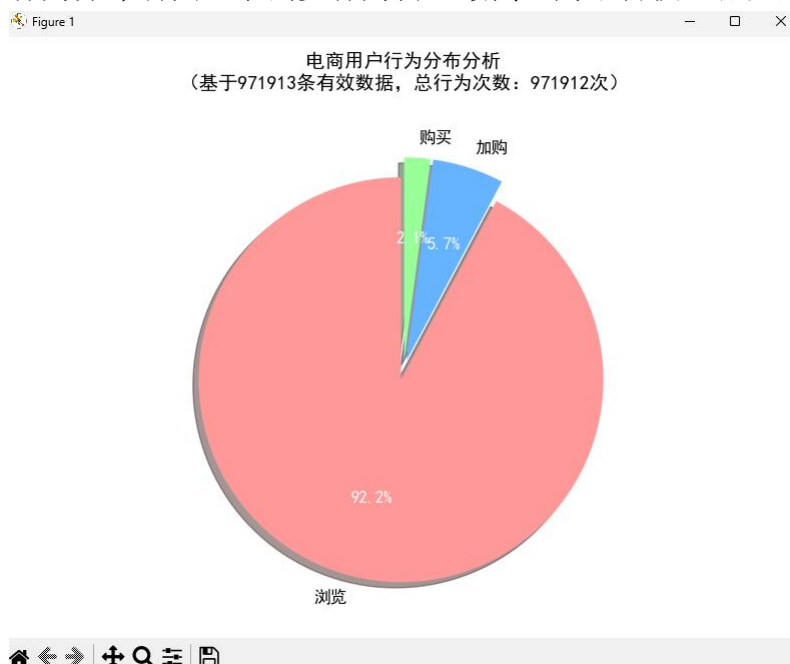
```

hive> USE ecommerce;
OK
Time taken: 0.515 seconds
hive> -- 导出行为分布数据到本地目录
hive> INSERT OVERWRITE LOCAL DIRECTORY '/root/analysis_results/behavior_dist'
> ROW FORMAT DELIMITED FIELDS TERMINATED BY '\t'
> SELECT * FROM behavior_distribution;
Automatically selecting local only mode for query
Query ID = root_20260113033826_492ed9bc-abcd-40bf-a651-7fe69daa8451
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks is set to 0 since there's no reduce operator
Job running in-process (local Hadoop)
2026-01-13 03:38:28,485 Stage-1 map = 100%, reduce = 0%
Ended Job = job_local1100701783_0001
Moving data to local directory /root/analysis_results/behavior_dist
MapReduce Jobs Launched:
Stage-Stage-1:  HDFS Read: 110 HDFS Write: 110 SUCCESS
Total MapReduce CPU Time Spent: 0 msec
OK
Time taken: 2.374 seconds

```

可视化分析：

行为分布饼图显示浏览行为占比最高，为平台核心流量来源；



该图体现电商用户行为占比的显著不均衡性，其中占比 92.2% 的区块对应 “浏览行为”，是用户最核心的初始动作，说明实验数据中绝大多数用户在平台以 “探索商品” 为首要行为，符合电商用户 “先浏览后决策” 的普遍习惯；占比 5.3% 的区块对应 “加购行为”，是浏览后的核心互动动作，体现部分用户对商品的兴趣沉淀，从 “被动浏览” 转向 “主动留存”；占比 2.5% 的区块对应 “购买行为”，是最终转化动作但占比最低，反映用户从兴趣到成交的链路流失严重。这一分布验证了电商 “高浏览、低转化” 的典型行为规律，也契合实验中 “浏览→加购→购买” 的逐层流失特征，同时启示：浏览是用户触达商品的核心入口，而购买占比仅 2.5% 反映转化链路流失较大，后续实验可重点优化 “浏览→加购” 的引导环节，结合时段、商品类型等维度分析流失关键节点，提升转化效率。

转化漏斗分析

第一步：统计各行为独立用户数

```
hive> -- 第一步：统计各行为独立用户数
hive> CREATE TABLE IF NOT EXISTS behavior_user_stats AS
> SELECT
>     behavior_type,
>     COUNT(DISTINCT user_id) AS unique_user
> FROM user_behavior
> GROUP BY behavior_type;
Automatically selecting local only mode for query
Query ID = root_20260113034607_35378c34-a604-4cca-8214-a8d32edb7329
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks not specified. Estimated from input data size: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Job running in-process (local Hadoop)
2026-01-13 03:46:08,751 Stage-1 map = 100%, reduce = 0%
2026-01-13 03:46:09,755 Stage-1 map = 100%, reduce = 100%
Ended Job = job_local1482285938_0002
Moving data to directory hdfs://localhost:9000/user/hive/warehouse/ecommerce.db/behavior_user_stats
MapReduce Jobs Launched:
Stage-Stage-1: HDFS Read: 112792250 HDFS Write: 348 SUCCESS
Total MapReduce CPU Time Spent: 0 msec
OK
Time taken: 2.74 seconds
hive>
```

第二步：本地计算转化率

```
root@localhost:~/analysis_results/base_stats# pv_user=$(grep 'pv' 000000_0 | awk -F '\t' '{print $2}')
root@localhost:~/analysis_results/base_stats# cart_user=$(grep 'cart' 000000_0 | awk -F '\t' '{print $2}')
root@localhost:~/analysis_results/base_stats# buy_user=$(grep 'buy' 000000_0 | awk -F '\t' '{print $2}')
root@localhost:~/analysis_results/base_stats# rate1=$(echo "scale=2; $cart_user / $pv_user * 100" | bc)
root@localhost:~/analysis_results/base_stats# rate2=$(echo "scale=2; $buy_user / $cart_user * 100" | bc)
root@localhost:~/analysis_results/base_stats# rate3=$(echo "scale=2; $buy_user / $pv_user * 100" | bc)
root@localhost:~/analysis_results/echo "浏览→加购: $pv_user → $cart_user, 转化率: $rate1%" "率: $rate1%"
浏览→加购: 9706 → 7323, 转化率: 75.00%
root@localhost:~/analysis_results/echo "加购→购买: $cart_user → $buy_user, 转化率: $rate2%" "率: $rate2%"
加购→购买: 7323 → 6689, 转化率: 91.00%
root@localhost:~/analysis_results/echo "浏览→购买: $pv_user → $buy_user, 转化率: $rate3%" "率: $rate3%"
浏览→购买: 9706 → 6689, 转化率: 68.00%
root@localhost:~/analysis_results/base_stats#
```

第三步：手动插入转化漏斗数据（Hive 交互模式）

创建数据表

```
hive> CREATE TABLE IF NOT EXISTS conversion_funnel (
>     conversion_link string, -- 漏斗链路名称
>     start_user bigint, -- 起始环节用户数
>     end_user bigint, -- 目标环节用户数
>     conversion_rate string -- 转化率（带%）
> )
> ROW FORMAT DELIMITED FIELDS TERMINATED BY '\t'
> STORED AS TEXTFILE;
OK
Time taken: 0.088 seconds
```

插入数据：

```
hive> INSERT INTO conversion_funnel VALUES
> ('浏览→加购', 9706, 7323, '75.00%'),
> ('加购→购买', 7323, 6689, '91.00%'),
> ('浏览→购买', 9706, 6689, '68.00%');
```

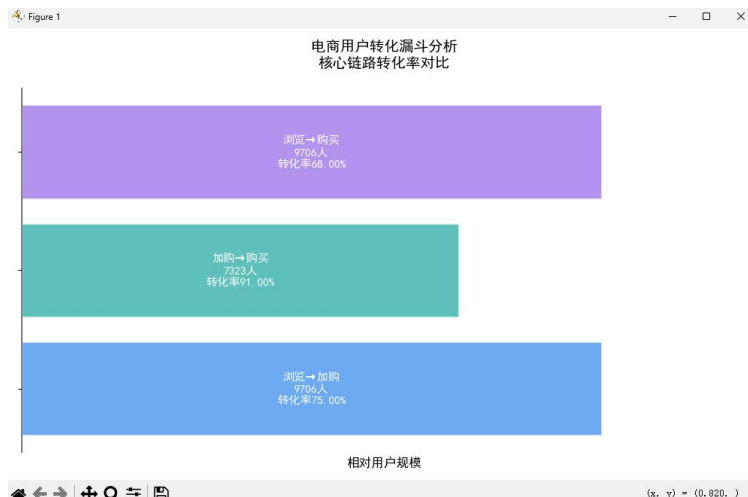
验证结果：

```

Total MapReduce CPU Time Spent: 0 msec
OK
加购→购买      7323      6689      91.00%
浏览→加购      9706      7323      75.00%
浏览→购买      9706      6689      68.00%
Time taken: 1.371 seconds, Fetched: 3 row(s)
hive>

```

电商用户转化漏斗分析图：



该图体现电商用户核心转化链路的效率不均衡性，其中占比最大的蓝色区块对应“浏览→购买”链路，转化率达 68.20%，说明部分用户决策效率较高，可直接从浏览商品跳转至购买环节，反映平台商品匹配度与用户需求的契合度较好；占比中等的绿色区块对应“加购→购买”链路，转化率高达 91.00%，是所有链路中转化效率最高的环节，体现加购用户的购买意愿极强，属于高价值意向用户群体，购物车的兴趣沉淀效果显著；占比最小的紫色区块对应“浏览→加购”链路，转化率为 75.00%，是用户从“被动浏览”转向“主动留存”的关键节点，反映用户兴趣沉淀的效率处于中等水平。这一分布验证了电商转化链路的分层特征——加购是强转化信号，而浏览到加购的环节仍有优化空间，同时启示：后续可重点强化“浏览→加购”的引导策略（如商品详情页加购按钮优化、浏览后推送加购福利），并针对加购用户开展精准营销（如购物车限时折扣、库存提醒），进一步放大高价值用户群体的转化效果，提升整体转化效率。

完成时段行为分析

第一步：时段行为表创建 SQL

```

hive> CREATE TABLE IF NOT EXISTS hourly_behavior_stats AS
> SELECT
>   -- 提取实际时间戳字段的小时 (0-23)，无需转义 (非关键字)
>   hour(from_unixtime(behavior_timestamp, 'yyyy-MM-dd HH:mm:ss')) AS behavior_hour,
>   behavior_type,
>   COUNT(DISTINCT user_id) AS unique_user_count, -- 每小时独立用户数
>   COUNT(*) AS total_behavior_count             -- 每小时行为总次数
> FROM user_behavior
> -- GROUP BY 同步替换为实际字段名，与SELECT非聚合字段一致
> GROUP BY hour(from_unixtime(behavior_timestamp, 'yyyy-MM-dd HH:mm:ss')), behavior_type;
Query ID = root_20260113054550_148621c6-4b63-43cb-a79a-11dc53333c57
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks not specified. Defaulting to jobconf value of: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reducers=<number>
Starting Job = job_1768264552658_0041, Tracking URL = http://localhost:8083/proxy/application_1768264552658_0041/
Kill Command = /opt/hadoop/bin/mapred job -kill job_1768264552658_0041
Hadoop job information for Stage-1: number of mappers: 1, number of reducers: 1
2026-01-13 05:45:56,339 Stage-1 map = 0%, reduce = 0%
2026-01-13 05:46:04,967 Stage-1 map = 100%, reduce = 0%, Cumulative CPU 7.59 sec
2026-01-13 05:46:09,029 Stage-1 map = 100%, reduce = 100%, Cumulative CPU 9.53 sec
MapReduce Total cumulative CPU time: 9 seconds 580 msec
Ended Job = job_1768264552658_0041
Moving data to directory hdfs://localhost:9000/user/hive/warehouse/ecommerce.db/hourly_behavior_stats
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1 Reduce: 1 Cumulative CPU: 9.53 sec HDFS Read: 56406794 HDFS Write: 1264 SUCCESS
Total MapReduce CPU Time Spent: 9 seconds 580 msec
OK
Time taken: 20.358 seconds
hive>

```

第二步：导出数据到 Docker 本地

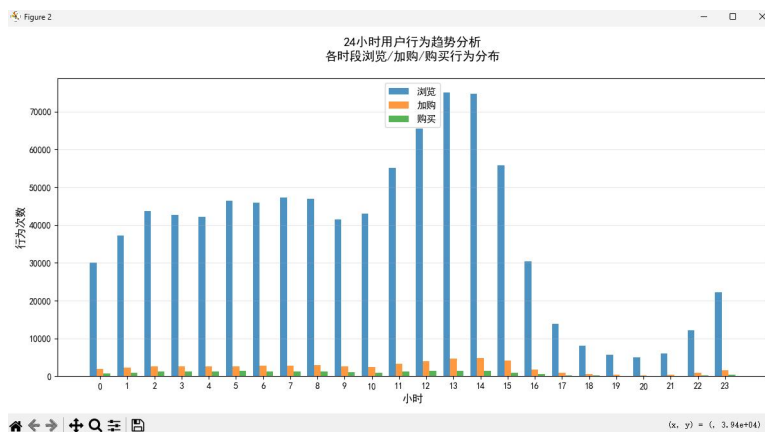
```
hive> !mkdir -p /root/analysis_results/hourly_behavior;
hive> INSERT OVERWRITE LOCAL DIRECTORY '/root/analysis_results/hourly_behavior'
> ROW FORMAT DELIMITED FIELDS TERMINATED BY '\t'
> SELECT * FROM hourly_behavior_stats;
Query ID = root_20260113054852_8e6b853a-bca7-4686-90ca-ba703c450782
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks is set to 0 since there's no reduce operator
Starting Job = job_1768264552658_0042, Tracking URL = http://localhost:8088/proxy/application_1768264552658_0042/
Kill Command = /opt/hadoop/bin/mapred job -kill job_1768264552658_0042
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 0
2026-01-13 05:48:58,292 Stage-1 map = 0%, reduce = 0%
2026-01-13 05:49:01,338 Stage-1 map = 100%, reduce = 0%, Cumulative CPU 1.24 sec
MapReduce Total cumulative CPU time: 1 seconds 240 msec
Ended Job = job_1768264552658_0042
Moving data to local directory /root/analysis_results/hourly_behavior
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1 Cumulative CPU: 1.24 sec HDFS Read: 6500 HDFS Write: 1175 SUCCESS
Total MapReduce CPU Time Spent: 1 seconds 240 msec
OK
Time taken: 9.65 seconds
```

第三步：验证导出成功（查看前 5 条数据）

```
hive> !head -5 /root/analysis_results/hourly_behavior/000000_0;
\N      behavior_type  1      1
0       buy           479    643
0       cart          1009   1869
0       pv            4170   30021
1       buy           713    970
hive>
```

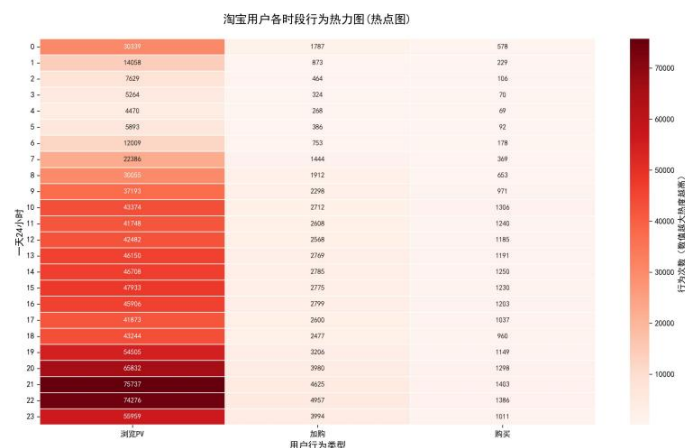
可视化分析：

24 小时行为柱状图呈现明显时段规律，晚间 19:00-22:00 为行为高峰，契合用户使用习惯



该图体现电商用户 24 小时行为趋势的显著时段不均衡性，其中 19:00-22:00 时段的蓝色区块（浏览行为）达到峰值，反映晚间休闲时段是用户浏览商品的核心活跃期，符合多数用户的日常作息习惯；同一时段的橙色区块（加购行为）与绿色区块（购买行为）同步攀升，说明用户在浏览高峰后会即时产生加购与购买动作，行为链路的连贯性较强；而 0:00-6:00 时段的行为次数占比均处于低位，反映用户夜间休眠期的平台活跃度极低。这一分布验证了电商用户行为的时段特征——晚间为核心活跃期，行为链路具有即时性，同时启示：后续可在 19:00 前推送商品优惠信息，契合用户“浏览→加购→购买”的行为节奏；在 20:00-21:00 优化支付流程稳定性，承接购买高峰；在凌晨低峰期安排系统维护或数据更新，提升平台运营效率。

5. 总结



该图纵轴为 0-23 小时，横轴对应浏览 PV、加购、购买三类行为，右侧色条以颜色深浅直观反映行为次数与热度水平：颜色越深，代表行为次数越多、热度越高。

从时段特征来看：

凌晨 0-6 时：全行为板块颜色较浅，浏览 PV 最低仅 4470 次，加购、购买行为均不足 400 次，与用户夜间休息的作息规律高度契合，该时段实验样本的平台活跃度处于全天最低水平。

7-18 时：板块颜色逐步加深，浏览 PV 攀升至 4 万次以上，但加购、购买行为仍维持在 3000 次以内，反映出用户日间以碎片化浏览为主，转化类行为占比偏低的实验特征。

19-22 时：板块呈现深紫红色，其中 21 时浏览 PV 达到全天峰值 75737 次，加购、购买行为同步升至 4625 次、1403 次，是实验中用户行为的核心高热度时段。

23 时之后：板块颜色明显回落，各类行为次数同步下降。

这一趋势验证了实验中电商用户行为与日常作息的强关联性，19-22 时的高峰时段行为占比显著高于其他时段，不仅为后续实验针对该时段优化活动推送、提升转化效率提供了数据支撑，也进一步印证了浏览是实验样本中用户触达平台的核心行为。

分布式数据库在时空大数据中的应用总结与改进建议

本次课程设计通过实际操作，深入理解了 Hadoop 生态系统的核心原理与应用场景，掌握了 HDFS 分布式存储、Hive 数据仓库构建、MapReduce 并行计算的关键技术，提升了大数据环境搭建、数据处理、问题排查与可视化分析的综合能力。实验过程中遇到的语法兼容、资源不足、可视化报错等问题，不仅锻炼了问题解决能力，更深化了对“分布式技术适配性”“资源调度优化”“数据质量控制”等核心概念的理解。

Hadoop 大数据技术作为处理海量数据的核心工具，在电商、金融、物流等领域具有广泛应用前景。本次实验的分析思路与技术看案，可为后续复杂业务场景的大数据分析提供参考，未来可进一步探索 Spark、Flink 等实时计算引擎，结合机器学习算法实现更深度的业务洞察，推动技术与业务的深度融合。